



universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Software Architectures

Architecture Checkpoint Report

Assignment 2: Architectural Evolution of nopCommerce

Group 107

Afonso Ferreira	113480
Tomás Brás	112665
Hugo Ribeiro	113402
Rodrigo Abreu	113626

April 30, 2026

Contents

1	Introduction and Scenario	1
1.1	Scenario Choice	1
1.2	Core Architectural Problem	1
2	Current-State Analysis	2
2.1	Architecture Overview	2
2.2	How a Purchase Works Today	3
2.3	What Already Works for Scenario C	3
2.3.1	Order Capture and Customer Visibility	3
2.3.2	Inventory and Warehouse Model	3
2.3.3	Fulfillment and Shipping	4
2.3.4	Plugins and Scheduled Tasks	4
2.4	What Does Not Work for Scenario C	4
2.4.1	No Durable Integration Path	4
2.4.2	No Freshness or Source Metadata on External State	4
2.4.3	Synchronous Plugin Calls Expose the Customer to External Failures	4
2.4.4	No Standard Failure State or Reconciliation Model	4
2.4.5	Shared Database Must Stay Internal	4
2.5	Useful Seams for Evolution	5
2.6	What This Means for the Target Architecture	5
3	Domain and Bounded Contexts	6
3.1	Domain Overview	6
3.2	Relevant Subdomains	7
3.3	Bounded Contexts	7
3.3.1	nopCommerce Commerce Core Context	7
3.3.2	Integration Coordination Context	8
3.3.3	Warehouse Fulfillment Context	8
3.3.4	Inventory Availability Context	9
3.3.5	Shipping Context	9
3.3.6	Store Operations / POS Context	10
3.3.7	Support and Operations Context	10
3.4	Responsibilities	11
3.5	Boundary Model	11
3.6	Boundary Rules	12
3.7	Architecture Diagram	12
4	Quality Attribute Scenarios	13
4.1	Architectural Drivers	13
4.2	Scenario 1 - Availability under Warehouse Failure	13
4.3	Scenario 2 - Consistency under Stale Inventory	14

4.4	Scenario 3 - Performance during Omnichannel Order Bursts	14
4.5	Scenario 4 - Recoverability after Shipping Outage	15
4.6	Scenario 5 - Modifiability for New Operational Channels	15
4.7	Traceability to Architectural Drivers	16
5	Architectural Approach	17
5.1	Chosen Framework	17
5.2	Justification	17
5.3	Why Not ACDM or ADM as the Main Framework	18
5.4	How ADD Shapes the Target Architecture	18
6	Target Architecture	19
6.1	Overview	19
6.2	Main Components and Services	19
6.3	Synchronous and Asynchronous Interactions	20
6.4	Data Ownership	20
6.5	Cross-Cutting Concerns	20
6.6	Main Architecture Diagram	21
7	Architectural Decisions	22
7.1	ADR 1 — Asynchronous Fulfillment Propagation	22
7.1.1	Context	22
7.1.2	Decision	22
7.1.3	Alternatives Considered	22
7.1.4	Consequences	23
7.2	ADR 2 — Transactional Outbox for Durable Integration Task Publishing	23
7.2.1	Context	23
7.2.2	Decision	23
7.2.3	Alternatives Considered	23
7.2.4	Consequences	24
7.3	ADR 3 — Adapter Pattern with No Shared Database Across External Boundaries	24
7.3.1	Context	24
7.3.2	Decision	24
7.3.3	Alternatives Considered	24
7.3.4	Consequences	25
7.4	ADR 4 — nopCommerce Monolith Retained as the Commerce Core	25
7.4.1	Context	25
7.4.2	Decision	25
7.4.3	Alternatives Considered	26
7.4.4	Consequences	26
8	Risk and Validation Plan	27
8.1	Purpose	27
8.2	Main Architectural Risks	27
8.3	Validation Activities	28
8.4	Feasibility Spike Focus	28
9	Evolution Roadmap	29
9.1	Implementation Order	29
9.1.1	Phase 1 — Outbox and Asynchronous Propagation	29
9.1.2	Phase 2 — Warehouse Adapter and Degraded-State Handling	29
9.1.3	Phase 3 — Inventory Projection and Stale-State Visibility	30

9.1.4 Phase 4 — Operational Visibility	30
10 Feasibility Spike	31
10.1 What Was Tested	31
10.2 What Worked	31
10.3 Evidence	31

Chapter 1

Introduction and Scenario

1.1 Scenario Choice

This project follows **Scenario C - Omnichannel Commerce Core**. VerdeMart Retail started by using nopCommerce as its online storefront, but the business now needs more than a web shop. It wants web sales, warehouse execution, shipping, store operations, support, and customer visibility to work together as part of one commerce experience.

In this scenario, nopCommerce becomes the commerce core of that wider ecosystem. A web order should be reflected in operational systems such as warehouse and shipping, and changes that happen outside nopCommerce, such as stock or fulfillment updates, should become visible again in the customer and operator experience. This is important because those surrounding systems may be slow, stale, unavailable, or inconsistent, but the commerce experience still needs to remain useful and understandable.

1.2 Core Architectural Problem

Currently, nopCommerce mainly acts as the online shop: it owns the storefront, catalog browsing, cart, checkout, customer accounts, and order creation inside one web-centered platform. This works while the business is mostly operating through the online channel, but it does not fully address an omnichannel environment where warehouse execution, shipping, store operations, and support systems also influence the customer experience.

The core architectural problem is deciding how this online shop can evolve into a commerce core without assuming that every surrounding operational system is always fast, available, and consistent. The architecture must define what remains inside nopCommerce, what moves around it, and how the system behaves when warehouse or shipping state is delayed, stale, unavailable, or later recovered.

Chapter 2

Current-State Analysis

2.1 Architecture Overview

nopCommerce follows onion architecture, where all dependencies point inward toward the core. The solution is split into four main layers: **Nop.Core**, **Nop.Data**, **Nop.Services**, and the presentation layer (**Nop.Web** and **Nop.Web.Framework**). Each layer has a specific role:

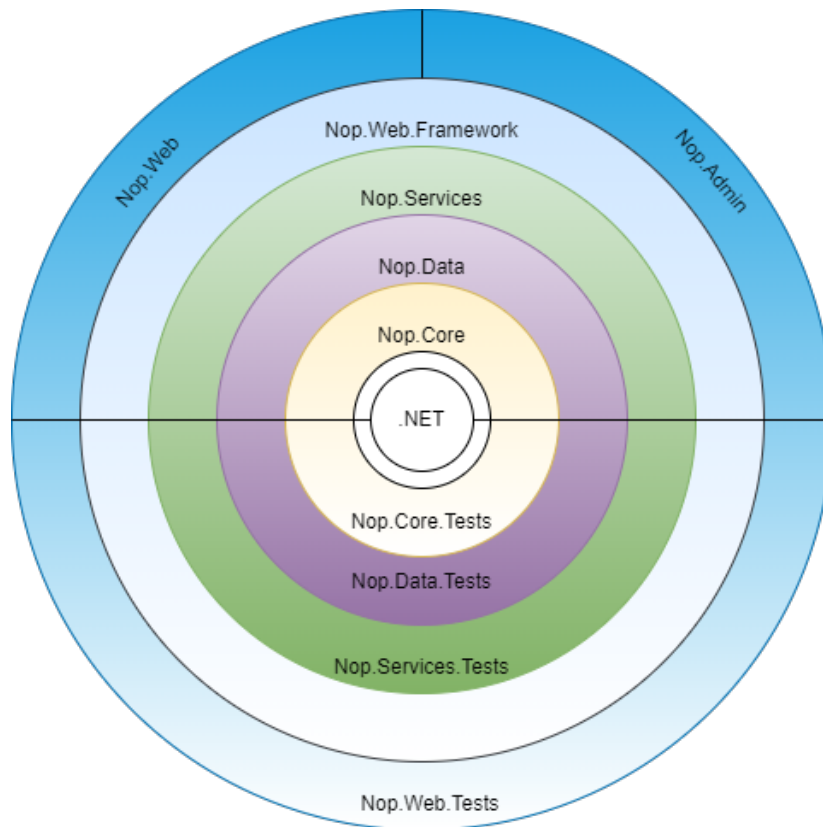


Figure 2.1: Current layered/onion architecture of nopCommerce.

- **Nop.Core:** The innermost layer. Contains domain entities, caching, events, and helper classes shared across the whole solution. It has no dependencies on any other project. Key Scenario C entities live here: **Order**, **OrderItem**, **Product**, **ProductWarehouseInventory**, **StockQuantityHistory**, **Warehouse**, and **Shipment**.
- **Nop.Data:** Separates data-access logic from business objects. nopCommerce uses a Linq2DB Code-First approach, with FluentMigrator for schema migrations and Fluent API for

persistence mappings.

- **Nop.Services:** The outermost layer of the Application Core. It holds business logic, validations, and calculations, and uses a repository pattern to expose features to layers above it. This is where order processing, inventory adjustment, shipping, and checkout logic live.
- **Nop.Web** and **Nop.Web.Framework:** The presentation layer. **Nop.Web** contains controllers, models, views, and the storefront and admin area. **Nop.Web.Framework** is a shared class library with common presentation logic used by both the public site and the admin panel.
- **Plugins:** Loaded as separate Visual Studio projects and extend nopCommerce for payments, shipping, authentication, search, and other integrations. They run inside the same runtime as the monolith.

2.2 How a Purchase Works Today

A customer order flows from the storefront controllers in **Nop.Web** down through **Nop.Services**. **OrderProcessingService** creates the order, adjusts inventory, persists state via repositories, and publishes internal events like **OrderPlacedEvent**. Shipment updates follow later through shipment services, which update order status and trigger notification events. This works cleanly when nopCommerce owns the full picture. It breaks down when an external warehouse, POS, or carrier system owns part of the order's lifecycle and may be slow, unavailable, or return conflicting data. The internal event system is also worth noting. It publishes events in-process to registered consumers and awaits each one. If a consumer fails, the error is logged and execution continues. There is no message broker and no standard durable integration retry, replay, or dead-letter mechanism. This is fine for cache invalidation or plugin reactions — it is not sufficient for reliable integration with external systems.

2.3 What Already Works for Scenario C

The current implementation already provides several Scenario C-relevant capabilities inside the monolith: web checkout and order creation, a local stock and warehouse model, shipment and tracking records, customer/admin order visibility, and plugin or scheduled-task extension points.

2.3.1 Order Capture and Customer Visibility

nopCommerce has mature order management: customer accounts, checkout, payment status, order history, shipment tracking, admin screens, and notification emails. This gives Scenario C a solid foundation. The order placement workflow is a natural seam where an integration event like "order accepted for fulfillment" can be emitted — currently it only fires in-process, but extending it with a durable outbox is straightforward without touching checkout itself.

2.3.2 Inventory and Warehouse Model

The platform already models stock quantity, reserved quantities, warehouse assignments, and stock history. This is directly useful for omnichannel stock visibility. The limitation is that nopCommerce treats this as local application state — it has no concept of whether a stock number came from the WMS ten seconds ago or ten hours ago, or whether a POS sale has already reduced it.

2.3.3 Fulfillment and Shipping

Shipment entities, tracking numbers, shipping plugins, pickup-in-store support, and customer notifications for shipping progress are all present. These support buy-online-fulfill-elsewhere flows. The gap is that shipment state is currently set by administrators or computed inside nopCommerce — there is no first-class integration status like "sent to WMS," "warehouse delayed," or "awaiting carrier confirmation."

2.3.4 Plugins and Scheduled Tasks

The architecture is modular, making it compatible with a variety of third-party services including ERPs, PIMs, and POS systems through its Web API plugin. Scheduled tasks can run background work — retries, synchronization, cleanup — and are protected against overlapping execution. These are useful extension points for Scenario C, though they lack the reliability structure needed for durable integration on their own.

2.4 What Does Not Work for Scenario C

2.4.1 No Durable Integration Path

The biggest gap is that there is no outbox, no integration message lifecycle, and no retry state. When an order is placed and the warehouse system is down, nopCommerce has no built-in guarantee that a fulfillment command was recorded, will be retried, or can be reconciled later. Logging an error is not a recovery path — operators need to know whether the order is waiting, failed, duplicated, or already accepted.

2.4.2 No Freshness or Source Metadata on External State

Stock, shipment, and fulfillment data can come from WMS, POS, ERP, or manual admin updates. The current model stores a value but does not record where it came from, when it was last confirmed, or whether it conflicts with another source. Stale data looks identical to fresh data to both customers and operators.

2.4.3 Synchronous Plugin Calls Expose the Customer to External Failures

When a plugin calls an external system during checkout or shipping-rate calculation, a slow or failed response degrades the customer experience directly. Scenario C requires the commerce core to stay useful when surrounding systems degrade — accepting safe work, signaling degraded status, and deferring unsafe steps — rather than failing visibly.

2.4.4 No Standard Failure State or Reconciliation Model

There is no common status model for integration messages: no "pending," "retrying," "failed," or "dead-lettered" states. There is no correlation between a placed order, the fulfillment command sent from it, and the acknowledgement (or conflict) that comes back. This makes recovery from partial failures opaque.

2.4.5 Shared Database Must Stay Internal

The nopCommerce database blends catalog, orders, customers, stock, shipments, configuration, and logs. This is fine for a monolith. It becomes a problem if external adapters start reading or writing it directly. Presentation logic in `Nop.Web` is isolated from the database — controllers only reference services, not repositories directly. The same discipline must apply at system boundaries:

any fulfillment adapter, stock projection, or integration worker must communicate through APIs or events, not by coupling to nopCommerce tables.

2.5 Useful Seams for Evolution

The architecture provides clear attachment points without requiring a rewrite:

- **Order placement:** emit a durable integration message for fulfillment and ERP immediately after an order is accepted.
- **Shipment updates:** receive warehouse and carrier progress and update nopCommerce shipment projections and customer-visible status.
- **Inventory:** receive stock changes from WMS or POS and update a customer-facing availability view with freshness metadata.
- **Scheduled tasks:** run outbox publishing, retry logic, timeout detection, and reconciliation outside the customer request path.
- **Plugins and APIs:** connect to surrounding systems through explicit adapters, not direct database access.
- **Admin visibility:** expose integration status, failures, and reconciliation actions to operators.

2.6 What This Means for the Target Architecture

nopCommerce should remain the owner of checkout, order management, customer visibility, catalog, and administration. Moving those out would create unnecessary scope. The evolution targets three specific gaps instead:

- **Reliable propagation:** fulfillment and ERP commands must leave through a durable asynchronous path, not in-process events or direct plugin calls.
- **Visible degradation:** when WMS, POS, ERP, or carriers are slow, unavailable, or contradictory, the system should expose an honest state — pending, delayed, degraded, or reconciliation required — rather than silently serving stale data.
- **Recovery and reconciliation:** retries, idempotency, correlation identifiers, and operator-visible failure records should make it possible to recover without duplicating orders or hiding conflicts.

The current state is a strong starting point: it provides a real commerce base with genuine seams. The evolution adds reliability and cross-channel state visibility around the existing monolith rather than replacing it.

Chapter 3

Domain and Bounded Contexts

3.1 Domain Overview

Scenario C changes the role of nopCommerce from an online storefront into the commerce core of VerdeMart’s wider omnichannel ecosystem. The domain is therefore not limited to product browsing and online checkout. It also includes fulfillment coordination, inventory visibility, store activity, shipping progress, support visibility, and recovery when surrounding systems are delayed, stale, unavailable, or contradictory.

The current nopCommerce baseline already contains important domain concepts for this scenario. **Nop.Core** defines entities such as **Order**, **OrderItem**, **Product**, **Warehouse**, and **Shipment**. It also includes warehouse inventory and stock history entities such as **ProductWarehouse Inventory** and **StockQuantityHistory**. **Nop.Services** contains the application logic around order placement, inventory adjustment, shipping, plugins, internal events, and scheduled tasks. **OrderProcessingService** persists the order, moves cart items into order items, adjusts inventory, sends notifications, and publishes an in-process **OrderPlacedEvent**. These are useful seams, but they are not yet a durable integration boundary for external warehouse, POS, shipping, or support systems.

The target model keeps nopCommerce as the owner of the customer-facing commercial transaction. It does not decompose the monolith into many services. Instead, it defines explicit boundaries around the operational capabilities that surround the commerce core. Warehouse execution, store/POS stock changes, shipping-provider state, and support operations visibility must communicate through APIs, events, or adapters. They must not read or write the nopCommerce database directly.

The main domain tension is ownership under imperfect information. nopCommerce can own the canonical web order, but it cannot assume that every external stock number, warehouse acknowledgement, or carrier update is immediately correct. The bounded contexts below make stale, pending, failed, retried, and reconciled states explicit so the system remains useful and explainable during degradation.

3.2 Relevant Subdomains

Subdomain	Type	Description
Customer commerce	Core	Product discovery, cart, checkout, payment result handling, customer order history, and the canonical web order.
Fulfillment coordination	Core / supporting	Propagates accepted orders to warehouse, store, ERP, and shipping capabilities through a durable target-state integration boundary.
Inventory availability	Supporting	Presents customer-facing availability using nopCommerce stock data and projected updates from warehouse and store/POS systems, including freshness and conflict state.
Warehouse fulfillment	External supporting	Owens picking, packing, dispatch preparation, fulfillment acknowledgement, and warehouse failure reasons.
Store operations / POS	External supporting	Owens in-store sale events, pickup activity, and local stock changes that must become visible again in the commerce experience.
Shipping	External generic	Owens label creation, carrier tracking updates, and carrier-specific failure responses.
Support and operations visibility	Supporting	Gives support agents and operators a view of pending, degraded, failed, retried, dead-lettered, and reconciled integration work.

3.3 Bounded Contexts

3.3.1 nopCommerce Commerce Core Context

This context remains inside nopCommerce. It owns catalog presentation, customer accounts, shopping carts, checkout, payment result handling, order creation, and customer-facing order history. The current codebase already supports these responsibilities through the existing layered/onion structure: domain entities in `Nop.Core`, persistence through `Nop.Data`, business logic in `Nop.Services`, and web/admin presentation through `Nop.Web` and `Nop.Web.Framework`.

For Scenario C, this context remains the system of record for the canonical web order. It may publish an order-accepted integration task in the target architecture, but it should not depend synchronously on warehouse, POS, or shipping providers during checkout. This supports availability and performance by keeping the customer request path separate from slow operational dependencies.

Owens:

- customer accounts, carts, checkout state, and payment result state;
- canonical web orders and order items;
- customer-facing order history and local order status;
- local catalog and stock concepts already represented by `Product`, warehouse inventory records, and stock quantity history.

Does not own:

- warehouse picking and packing execution;
- POS sales ledgers or store stock ledgers;
- carrier labels and carrier tracking source of truth;

- target-state retry queues, dead-letter records, or reconciliation records for external integration.

Boundary rule: external systems may receive commands or events derived from nopCommerce orders, but they must not access nopCommerce tables directly.

3.3.2 Integration Coordination Context

This is a target-state context introduced around nopCommerce, not a complete capability in the current baseline. Its responsibility is to make communication with surrounding systems durable, observable, and recoverable. It owns integration messages or outbox-style tasks, correlation identifiers, retry attempts, idempotency keys, failure states, dead-letter records, and reconciliation records.

The current in-process event publisher is useful inside the monolith, but it is not enough for reliable external integration because consumers run in the same process and failures are logged rather than retained as durable work. Scheduled tasks are a useful execution seam for polling, retry, timeout detection, or cleanup, but the target architecture still needs explicit integration state to know what is pending, duplicated, failed, or recovered.

Owns:

- target-state integration messages, outbox-style tasks, and delivery status;
- correlation identifiers linking a web order to warehouse, shipping, inventory, and support records;
- idempotency keys to prevent duplicate fulfillment or label creation during retries;
- retry, dead-letter, timeout, and reconciliation state.

Does not own:

- the canonical web order;
- warehouse operational execution;
- carrier tracking source of truth;
- POS sales.

Boundary rule: every external integration must pass through an adapter, API, or event contract. Shared database access is not an allowed shortcut.

3.3.3 Warehouse Fulfillment Context

This context represents the warehouse execution capability in Scenario C. It owns physical fulfillment state such as accepted for picking, picked, packed, delayed, failed, ready for dispatch, and warehouse acknowledgement. nopCommerce already has **Warehouse**, warehouse inventory, and shipment-related concepts, but the current baseline does not model a separate WMS lifecycle such as “sent to warehouse”, “warehouse delayed”, or “fulfillment rejected”.

In the target architecture, nopCommerce sends a fulfillment request asynchronously after order acceptance. The commerce core then receives a projection of warehouse state for customer, admin, and support visibility. This supports availability and recoverability because a slow or unavailable warehouse does not block checkout and can be retried or reconciled later.

Owns:

- picking, packing, dispatch preparation, and warehouse failure reasons;

- warehouse acknowledgement or rejection of fulfillment work;
- warehouse-side timing and operational exceptions.

Typical target-state messages:

- `FulfillmentRequested`;
- `FulfillmentAccepted`;
- `OrderPicked` or `OrderPacked`;
- `FulfillmentDelayed` or `FulfillmentFailed`.

3.3.4 Inventory Availability Context

This context provides cross-channel stock visibility for the web storefront, checkout, operators, and support agents. The baseline nopCommerce product model includes stock quantity, multiple-warehouse inventory records, reserved quantity, and stock history. These records are a strong starting point, but they do not distinguish fresh external stock from stale external stock, nor do they record contradictory source-system claims as first-class business states.

The target context treats inventory as a projection that includes source and freshness metadata. Warehouse and store/POS systems may report changes at different times. The web storefront may therefore show stock that is temporarily stale. Instead of hiding this as a technical problem, the model exposes states such as `InventoryStale` and `ReconciliationRequired`. This supports consistency by making inconsistency visible and recoverable rather than pretending all channels are immediately consistent.

Owns:

- customer-facing availability projections;
- source, last-confirmed time, stale-state, and conflict-state metadata in the target model;
- inventory discrepancy and reconciliation records.

Does not own:

- every physical stock movement in the warehouse or store;
- internal POS sale accounting;
- the canonical web order.

3.3.5 Shipping Context

This context represents shipping-provider or carrier integration. The baseline already has `Shipment`, tracking number, shipped date, delivered date, ready-for-pickup date, shipping plugins, and admin shipment operations. These support the target architecture, but they do not by themselves provide a durable label-request workflow with retry, idempotency, and provider-outage state.

In the target architecture, label creation and tracking updates are handled through a shipping adapter or provider API. If the provider is unavailable, the request remains pending or failed in the integration coordination context and is retried after recovery. nopCommerce may display shipping progress as a projection, while the carrier remains the owner of carrier-specific tracking state.

Owns:

- carrier label creation and label failures;

- carrier tracking updates;
- provider-specific shipping errors.

Typical target-state messages: `ShipmentReadyForLabel`, `LabelCreationRequested`, `LabelCreated`, `LabelCreationFailed`, and `TrackingUpdated`.

3.3.6 Store Operations / POS Context

This context represents physical store activity. nopCommerce already supports pickup-in-store concepts through order and shipment fields and the pickup plugin, but a full POS system remains outside the current baseline. In Scenario C, the POS/store context owns in-store sale events, store stock changes, pickup readiness, and pickup completion.

The important boundary is that the POS does not update nopCommerce inventory tables directly. It publishes or exposes store events through an adapter. The inventory availability context can then update the commerce-facing projection, and the support context can expose any stale or conflicting state.

Owns:

- POS sale events and local store stock changes;
- pickup readiness and pickup completion events;
- store-side operational exceptions.

Does not own:

- online checkout;
- canonical web order creation;
- nopCommerce database tables.

3.3.7 Support and Operations Context

This context gives operators and support agents visibility into omnichannel state. The current nopCommerce admin area already exposes orders, shipments, warehouses, stock history, plugins, and scheduled tasks. The target architecture extends this with operational read models for integration state: pending, retrying, delayed, failed, dead-lettered, recovered, and requiring reconciliation.

This context matters because a resilient integration is still operationally weak if people cannot explain what happened to an order or stock change. Degraded states are therefore part of the business model, not only log messages.

Owns:

- operational status views and support-facing explanations;
- failure visibility, retry history, and reconciliation actions;
- audit trail of cross-boundary integration attempts in the target model.

Does not own:

- warehouse, store, or shipping execution;
- the canonical web order;
- external stock or carrier ledgers.

3.4 Responsibilities

Bounded context	Primary responsibilities	Key data owned
nopCommerce Commerce Core	Catalog presentation, cart, checkout, payment result handling, canonical order creation, customer/admin order visibility.	Customers, carts, checkout state, payment result state, canonical web orders, order items, local stock records.
Integration Coordination	Target-state durable communication, retry, idempotency, correlation, failure state, dead-letter handling, and reconciliation.	Integration messages, correlation IDs, retry attempts, idempotency keys, failure and reconciliation records.
Warehouse Fulfillment	Picking, packing, dispatch preparation, fulfillment acknowledgement, warehouse delay or failure reporting.	Warehouse fulfillment state, warehouse acknowledgement, picking and packing status.
Inventory Availability	Cross-channel availability projection, source and freshness metadata, stale-state detection, conflict detection, reconciliation.	Availability projection, stock-source metadata, freshness state, discrepancy records.
Shipping	Label creation, carrier tracking updates, provider error reporting.	Label request state, tracking number, carrier status, provider error state.
Store Operations / POS	Store sales, in-store stock changes, pickup readiness, pickup completion.	POS sale events, store stock events, pickup events.
Support and Operations	Operational visibility, degraded-state explanation, retry monitoring, reconciliation monitoring.	Operational read model, failure records, status transition history, integration audit trail.

3.5 Boundary Model

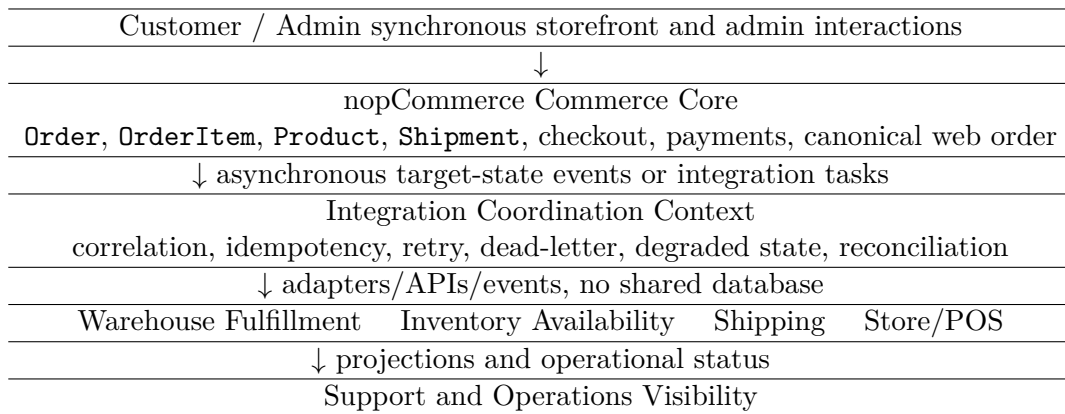
Boundary	Direction	Interaction style	Reason
Customer/admin to nopCommerce	Inbound	Synchronous	Browsing, checkout, and order visibility require immediate responses.
nopCommerce to integration coordination	Outbound	Asynchronous target-state event or task	Order propagation must survive warehouse or shipping delay without blocking checkout.
Integration coordination to warehouse adapter	Outbound	Async command/API call with retry	Fulfillment requests may be delayed or unavailable and must remain recoverable.
Warehouse adapter to integration coordination	Inbound	Event/callback/poll result	Picked, packed, delayed, failed, or accepted state must become visible.
Store/POS to inventory adapter	Inbound	Event/API update	Store stock changes should update availability without direct database coupling.
Integration coordination to shipping adapter	Outbound	Async command/API call with retry	Label creation and carrier calls can fail and must be idempotent.
Integration coordination to support view	Internal	Read model/projection	Operators need visibility into pending, degraded, failed, and recovered work.

3.6 Boundary Rules

1. **nopCommerce owns the canonical web order.** External systems may receive order commands or events, but they do not become the source of truth for the web order.
2. **No external system reads or writes the nopCommerce database directly.** This keeps the monolith's database internal and avoids hidden coupling across service boundaries.
3. **Checkout does not synchronously depend on warehouse, POS, or shipping.** External work is recorded and processed asynchronously in the target architecture.
4. **External operational state is projected back into the commerce experience.** Fulfillment status, shipping status, and stock availability can be shown to customers, administrators, or support agents, but they are projections with source and freshness metadata where needed.
5. **Every cross-boundary message needs correlation and idempotency.** This prevents duplicate fulfillment or duplicate label creation when retries happen after timeout, failure, or uncertain acknowledgement.
6. **Degraded states are domain states.** States such as pending warehouse confirmation, warehouse delay, awaiting shipping label, stale inventory, and reconciliation required should be visible to the appropriate customer, support, or operator view rather than hidden only in logs.
7. **Adapters translate external language into internal language.** Warehouse, POS, ERP, and carrier terms should not leak directly into the core order model.

3.7 Architecture Diagram

The following textual diagram summarizes the boundary model without introducing additional diagramming dependencies into the report:



This model is traceable to the quality attributes in Chapter 4. Availability is supported by accepting and persisting web orders without synchronously requiring the warehouse or shipping provider. Consistency is handled by explicit stale and conflicting inventory states rather than by assuming immediate agreement across channels. Performance is protected by keeping slow integration work outside the checkout request path. Recoverability is supported by retry, idempotency, correlation, and reconciliation in the target integration context. Modifiability is supported by adapters and by forbidding shared database access across external boundaries.

Chapter 4

Quality Attribute Scenarios

Quality attribute scenarios make the architectural drivers measurable. For the VerdeMart omnichannel scenario, the most important pressure is not only that nopCommerce must create orders correctly, but that the commerce experience must remain understandable while warehouse, shipping, store, and support systems are slow, stale, unavailable, or later recovering.

The scenarios below use the six-part structure from the course material: stimulus source, stimulus, artifact, environment, response, and response measure.

4.1 Architectural Drivers

The scenarios are driven by four architectural concerns:

- **Availability:** web orders must not be lost when warehouse or shipping systems fail.
- **Consistency:** inventory and order state may be temporarily stale, but inconsistencies must be visible and recoverable.
- **Performance:** customer-facing operations must stay responsive during backend synchronization and sales peaks.
- **Modifiability:** new operational channels must be integrated without rewriting checkout or sharing the nopCommerce database.

4.2 Scenario 1 - Availability under Warehouse Failure

Driver: Availability and operational continuity.

Part	VerdeMart scenario
Stimulus source	Warehouse execution system
Stimulus	The warehouse API takes more than 10 seconds to respond or rejects fulfillment requests for newly paid web orders.
Artifact	Fulfillment synchronization and the order status view shown to customers and operators
Environment	Friday afternoon peak traffic, with 3 times normal order volume, while the warehouse dependency is degraded but the web shop is still accepting orders
Response	The order is accepted and persisted locally; the fulfillment request is stored for asynchronous retry; the order is shown as awaiting warehouse confirmation; subsequent orders are not blocked by the degraded dependency.
Response measure	No paid order is lost; no duplicate fulfillment request is sent; pending fulfillment state is visible to support within 30 seconds; retry or escalation starts within 5 minutes.

4.3 Scenario 2 - Consistency under Stale Inventory

Driver: Consistency and recoverable inventory state.

Part	VerdeMart scenario
Stimulus source	Store point-of-sale system and warehouse stock service
Stimulus	A store sale reduces stock by 5 units, but the update reaches nopCommerce 3 minutes after the web catalog still showed those units as available.
Artifact	Inventory availability view used by product pages, cart validation, and checkout
Environment	Concurrent web and physical-store sales for the same product during a promotion, with external stock updates delayed by up to 3 minutes
Response	The commerce core marks stock data as stale, revalidates stock before checkout completion, preserves already-created orders for reconciliation, and records an inventory discrepancy for recovery.
Response measure	Product pages show stale stock status within 30 seconds of detection; overselling stays under 1% of affected order lines; inventory discrepancies are queued for reconciliation within 1 minute.

4.4 Scenario 3 - Performance during Omnichannel Order Bursts

Driver: Performance under customer-facing load.

Part	VerdeMart scenario
Stimulus source	Web customers, store operators, and customer support agents
Stimulus	A campaign creates 5 times normal traffic for product browsing, cart validation, order creation, and order-status queries.
Artifact	Storefront, checkout path, order status view, and integration event publishing path
Environment	Peak campaign period, with 5 times normal traffic and background synchronization still running
Response	The system prioritizes checkout writes, serves order-status reads from a projection, and processes warehouse and shipping synchronization asynchronously without blocking the customer request path.
Response measure	Checkout p95 latency remains below 2 seconds; order-status read p95 remains below 1 second; at least 99% of integration events are published or queued within 60 seconds.

4.5 Scenario 4 - Recoverability after Shipping Outage

Driver: Recoverability after external dependency failure.

Part	VerdeMart scenario
Stimulus source	Shipping provider or shipping adapter
Stimulus	Shipment label creation fails for 50 ready-to-dispatch orders and later becomes available again.
Artifact	Shipping integration workflow, retry queue, and order shipment status
Environment	Warehouse has packed orders, but the external shipping provider is unavailable for 20 minutes
Response	The system stores failed label requests, keeps orders in a clear awaiting-shipping state, avoids repeated synchronous calls from operators, and resumes processing automatically when the provider recovers.
Response measure	100% of failed label requests are retained; operators see the outage state within 30 seconds; 95% of queued labels are created within 10 minutes after provider recovery.

4.6 Scenario 5 - Modifiability for New Operational Channels

Driver: Modifiability and controlled integration growth.

Part	VerdeMart scenario
Stimulus source	VerdeMart business team and integration developers
Stimulus	A new store-operations system must receive order-ready-for-pickup events and send pickup-completed updates back to the commerce core.
Artifact	Integration layer, domain events, and order status update boundary
Environment	Existing web, warehouse, and shipping integrations are already in production
Response	The architecture allows the new channel to be added through a new adapter and event subscription without changing checkout logic or sharing the nopCommerce database with the external system.
Response measure	The new integration requires changes in no more than one adapter and one application-service boundary; no direct external database access is introduced; regression tests for checkout and existing integrations remain green.

4.7 Traceability to Architectural Drivers

Scenario	Architectural driver
Availability under warehouse failure	Availability, recoverability
Consistency under stale inventory	Consistency
Performance during omnichannel order bursts	Performance
Recoverability after shipping outage	Recoverability, availability
Modifiability for new operational channels	Modifiability

Chapter 5

Architectural Approach

5.1 Chosen Framework

The selected architectural design framework is **Attribute-Driven Design (ADD)**.

ADD fits this project because the VerdeMart scenario is mainly driven by measurable quality attribute pressures. The main question is not only which components should exist, but how the commerce core should behave when surrounding operational systems are delayed, stale, unavailable, or recovering.

In this project, ADD is applied as follows:

1. Drivers identified: availability, consistency, performance, recoverability, and modifiability.
2. Five quality attribute scenarios defined, each with a measurable response.
3. Tactics selected per scenario: asynchronous integration, transactional outbox-style event publishing, read-model separation, retry queue, reconciliation, adapter pattern, and observability.
4. Target architecture decomposed around those tactics: nopCommerce Core, Integration Layer, Operational Adapters, External Systems, and Support and Operations View.
5. Validation focused on the riskiest path: asynchronous propagation, degraded dependency handling, retry, and recovery.

5.2 Justification

The chosen scenario requires nopCommerce to evolve from a web storefront into the commerce core of a wider operational ecosystem. This creates architectural pressure in five main areas: availability, consistency, performance, recoverability, and modifiability.

ADD is appropriate because each of these pressures can be expressed as a concrete quality attribute scenario. Warehouse failure drives the need for asynchronous fulfillment and retry. Stale inventory drives the need for inventory visibility, revalidation, and reconciliation. Order bursts drive the need to protect customer-facing paths from slow backend synchronization. Shipping outage drives the need for durable retry and explicit recovery state. New operational channels drive the need for adapters and stable integration boundaries.

This gives a direct trace from the problem to the architecture:

ADD input	Scenario	Architectural consequence
Availability	Scenario 1	Avoid synchronous dependency on warehouse systems during checkout.
Consistency	Scenario 2	Introduce explicit inventory state, stale-state visibility, and reconciliation.
Performance	Scenario 3	Keep checkout and order reads separate from slow integration processing.
Recoverability	Scenario 4	Store failed integration work durably and retry after dependency recovery.
Modifiability	Scenario 5	Add external systems through adapters instead of direct database coupling.

5.3 Why Not ACDM or ADM as the Main Framework

ACDM was considered because it is useful for architectural decision-making, risk discussion, and stakeholder validation. However, it is not the main framework for this project because the strongest design input is the set of measurable quality attribute scenarios. The project must show runtime behavior under failure, delay, stale state, and recovery; ADD gives a more direct path from those scenarios to concrete architectural tactics and component responsibilities.

ACDM-style reasoning is still useful later in the project, especially when documenting ADRs and explaining rejected alternatives. For example, the decision to use asynchronous integration can be recorded as an ADR with alternatives such as direct synchronous calls or shared database access. However, the decision itself is primarily driven by the availability and recoverability scenarios, which makes ADD the better primary method.

ADM/TOGAF was also considered, but it is broader than necessary for this assignment. ADM is better suited to enterprise-wide transformation across business, data, application, and technology architecture. VerdeMart does have an omnichannel business context, but the scope of this work is a focused software architecture evolution of nopCommerce, not a full enterprise architecture programme.

5.4 How ADD Shapes the Target Architecture

Using ADD means that each architectural element in the target design should answer a specific quality attribute pressure. The surrounding systems are not added only because they exist in the business scenario; they are added because they create design pressure that nopCommerce must handle explicitly.

The target architecture emphasizes asynchronous integration because Scenario 1 requires that a warehouse timeout does not block checkout. A direct synchronous call from checkout to the warehouse cannot satisfy that constraint, so accepted orders are propagated through a durable integration path.

It also makes status explicit for pending, degraded, failed, and recovered work because Scenarios 1 and 4 require support agents and operators to understand what is happening during warehouse or shipping failure. Retry and reconciliation are part of the architecture because Scenarios 2 and 4 assume stale inventory, failed label creation, and later recovery. External systems are connected through adapters because Scenario 5 requires new operational channels to be added without rewriting checkout or sharing the nopCommerce database.

The next chapter uses these ADD-driven conclusions to define the target architecture.

Chapter 6

Target Architecture

6.1 Overview

The target architecture keeps nopCommerce as the commerce core, but stops treating every surrounding operational capability as a direct synchronous dependency. nopCommerce remains responsible for the customer-facing commerce experience: catalog browsing, cart, checkout, customer accounts, payment result handling, and order creation. Around it, the architecture adds integration boundaries for warehouse, inventory, shipping, store operations, and support visibility.

The main architectural change is the introduction of an integration layer between nopCommerce and the surrounding systems. This layer exposes adapters for external systems, publishes and consumes integration events, stores work that cannot be completed immediately, and retries or reconciles failed operations. This allows the commerce core to remain useful when warehouse, shipping, or store systems are slow, stale, unavailable, or recovering.

The architecture is therefore not a full microservice rewrite of nopCommerce. It is an evolutionary architecture where the monolith keeps the responsibilities that are already stable inside nopCommerce, while volatile omnichannel coordination is moved to explicit integration boundaries.

6.2 Main Components and Services

nopCommerce Commerce Core is the main application and remains the system of record for web customers, shopping carts, checkout, payments, and created orders. It owns the customer-facing transaction and decides whether an order can be accepted.

Integration Layer is responsible for communication between nopCommerce and external operational systems. It receives events from the commerce core, routes them to the correct adapter, stores pending integration work, and records failures for retry or reconciliation.

Warehouse Adapter translates accepted orders into fulfillment requests for the warehouse execution system. It also receives fulfillment updates such as picked, packed, or failed fulfillment.

Inventory Adapter synchronizes stock changes from warehouse and store systems into an inventory availability view used by nopCommerce. It is responsible for detecting stale or conflicting stock information and triggering reconciliation.

Shipping Adapter manages shipment label creation and shipping status updates. When the shipping provider is unavailable, it stores failed label requests and retries them later.

Store Operations Adapter represents physical-store capabilities such as pickup, in-store stock changes, and pickup-completed updates. It allows store operations to participate in the commerce flow without accessing the nopCommerce database directly.

Support and Operations View gives support agents and operators visibility over pending, degraded, failed, and recovered work. This is important because a degraded omnichannel system

must be understandable, not only technically resilient.

6.3 Synchronous and Asynchronous Interactions

The architecture uses synchronous interactions only when the user needs an immediate answer and the dependency is inside the commerce core. Product browsing, cart operations, checkout validation, payment result handling, and order creation remain synchronous from the customer's point of view. These operations must be fast and predictable because they directly affect the buying experience.

Interactions with warehouse, shipping, inventory synchronization, and store operations are primarily asynchronous. After an order is created, nopCommerce publishes an order-created event or stores an equivalent integration task. The integration layer then sends the fulfillment request to the warehouse adapter. If the warehouse is slow or unavailable, the order is not lost and the checkout path is not blocked; the fulfillment state becomes pending or degraded and is retried later.

Inventory updates also follow an asynchronous model. Warehouse and store systems publish stock changes, and the inventory adapter updates the availability view used by nopCommerce. Because this data may be stale, checkout still performs stock validation before completion, and detected conflicts are sent to reconciliation.

Shipping follows the same pattern. When the warehouse marks an order as ready to dispatch, the shipping adapter requests the label. If the provider fails, the label request is stored, the order is marked as awaiting shipping, and automatic retry resumes when the provider recovers.

6.4 Data Ownership

Data ownership is explicit to avoid hidden coupling between systems. nopCommerce owns customer accounts, carts, checkout state, payment result state, and the canonical web order. External systems are not allowed to read or write the nopCommerce database directly.

The warehouse system owns physical fulfillment state, such as picking, packing, dispatch preparation, and fulfillment failures. The shipping provider owns carrier label generation and carrier tracking updates. Store operations own in-store events such as pickup handling and point-of-sale stock changes. The integration layer owns integration metadata: pending work, retry attempts, correlation identifiers, failed messages, and reconciliation records.

nopCommerce may keep local views of external state, such as order fulfillment status, inventory availability, or shipping status, but these are projections used to support customer and operator visibility. They are not shared databases and should be updated through integration messages or adapter APIs.

6.5 Cross-Cutting Concerns

Reliability shapes the design because warehouse, shipping, and store systems can fail independently. Integration work must be durable, retries must be controlled, and repeated messages must not create duplicate fulfillment or shipping actions.

Observability is required because degraded state must be visible. Operators and support agents need to know whether an order is waiting for warehouse confirmation, shipping label creation, inventory reconciliation, or manual intervention.

Consistency is handled through explicit stale-state handling and reconciliation rather than by assuming immediate consistency across all channels. This is necessary because web, warehouse, and store operations can observe stock at different times.

Security and access control remain important because surrounding systems should not get direct database access. They interact through adapters, APIs, or events with controlled permissions.

Modifiability is supported by keeping external systems behind adapters. A new POS, warehouse, or shipping provider can be added by implementing or replacing an adapter without rewriting checkout or changing the core nopCommerce data model.

6.6 Main Architecture Diagram

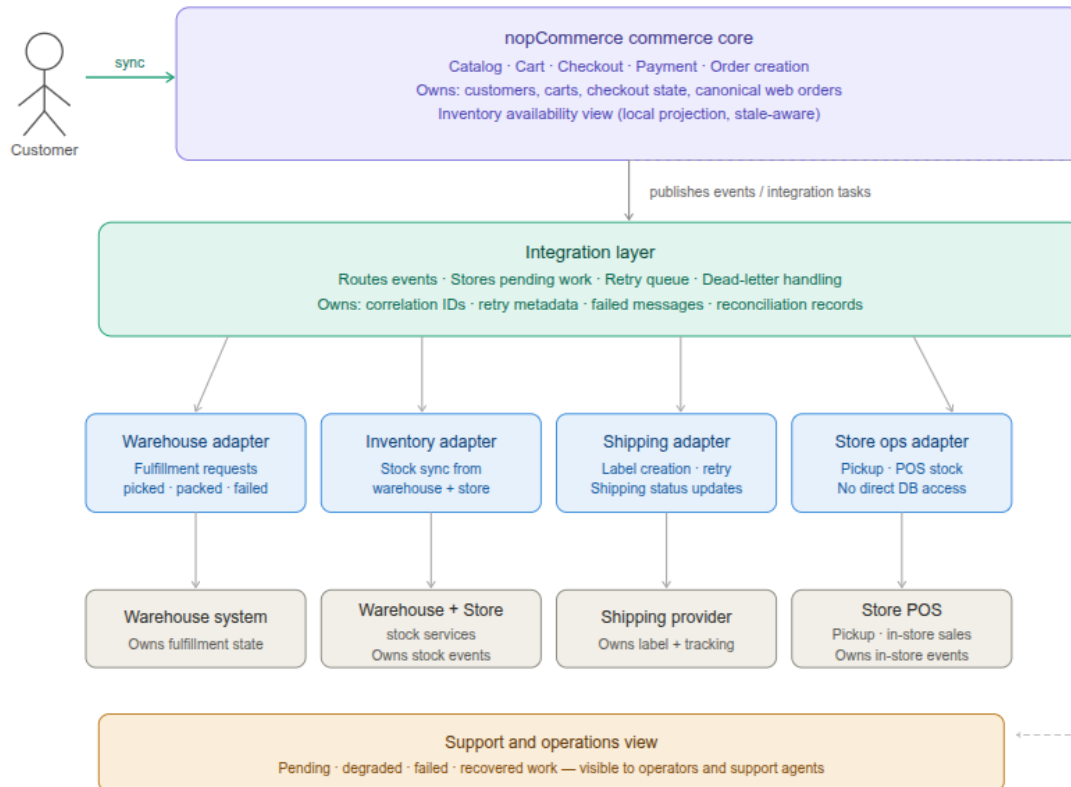


Figure 6.1: Target architecture for Scenario C: nopCommerce as an omnichannel commerce core.

The architecture diagram highlights the main design rule: customer-facing commerce remains inside nopCommerce, while operational coordination with warehouse, shipping, inventory, and store systems happens through explicit integration boundaries.

It also distinguishes customer-facing synchronous flows from asynchronous integration flows. This separation is important because checkout and order visibility must remain responsive even when external systems are slow or unavailable. Data ownership is explicit: nopCommerce owns customers, carts, checkout, payments, and canonical web orders; the integration layer owns retry metadata, correlation identifiers, dead letters, and reconciliation records; external systems own their operational state.

Chapter 7

Architectural Decisions

This chapter records the major architectural decisions made during the evolution of nopCommerce for the VerdeMart omnichannel scenario. Each decision is traceable to one or more quality attribute scenarios from Chapter 4 and to the bounded contexts defined in Chapter 3. Rejected alternatives are included with explicit reasons, since the value of a decision is inseparable from understanding what was considered and set aside.

7.1 ADR 1 — Asynchronous Fulfillment Propagation

7.1.1 Context

After a web order is accepted and paid, nopCommerce must notify the warehouse execution system. In the current baseline, external communication happens either through synchronous plugin calls or in-process events, both of which tie the checkout path directly to the availability of the downstream system.

QA Scenario 1 (Availability under Warehouse Failure) requires that a warehouse timeout or rejection does not prevent a web order from being recorded. Scenario 4 (Recoverability after Shipping Outage) further requires that failed fulfillment and shipping work is retained and retried, not silently dropped. A synchronous warehouse call from within the checkout flow cannot satisfy either constraint.

7.1.2 Decision

Fulfillment propagation is handled asynchronously. When nopCommerce accepts an order, it records an integration task for the fulfillment request. This task is picked up outside the customer request path, so a slow or unavailable warehouse has no effect on checkout completion. If the warehouse rejects or fails to acknowledge the request, the task stays pending and is retried by the Integration Coordination context.

Shipping follows the same model. Once the warehouse marks an order as ready to dispatch, the shipping adapter requests the carrier label independently. Provider unavailability results in a stored, retryable request rather than a visible checkout failure.

This is consistent with the boundary rule established in Chapter 3: checkout does not synchronously depend on warehouse, POS, or shipping systems.

7.1.3 Alternatives Considered

Synchronous API call to the warehouse during checkout. Calling the warehouse directly after order creation is straightforward to implement, but it makes checkout latency and success rate dependent on warehouse response times. During peak traffic, a warehouse response exceeding

ten seconds degrades the buying experience, and an outage blocks all order confirmations entirely. Scenario 1 rules this out.

In-process event bus (current nopCommerce approach). The existing `OrderPlacedEvent` fires in-process and awaits registered consumers. It works well for internal reactions like cache invalidation or plugin hooks, but it offers no durability for external integration. A process restart between order commit and event dispatch loses the event with no retry path. Chapter 2 identifies this limitation explicitly as a gap for the target scenario.

7.1.4 Consequences

Checkout becomes independent of warehouse and shipping availability, which is the primary goal. The trade-off is that fulfillment state is eventually consistent: the order model must expose intermediate states such as awaiting warehouse confirmation or fulfillment delayed, and those states must be visible to support agents and operators. Integration tasks also need idempotency guarantees to prevent duplicate fulfillment commands when retries occur, which is the subject of ADR 2.

7.2 ADR 2 — Transactional Outbox for Durable Integration Task Publishing

7.2.1 Context

ADR 1 requires that fulfillment work is propagated asynchronously, but that alone does not solve durability. A common approach publishes a message after the database transaction commits. The problem is the gap between the two operations: if the process restarts or the broker is unreachable at that moment, the integration task is lost while the order appears confirmed to the customer. The warehouse never receives the request, and without explicit detection, nobody knows.

The Integration Coordination context defined in Chapter 3 owns integration messages, correlation identifiers, idempotency keys, and retry state. For this ownership to be meaningful, there must be a reliable way to record integration work at the point the order is created.

7.2.2 Decision

Integration tasks are written in the same database transaction as the order. When `OrderProcessingService` creates an order, it also inserts an outbox record in the same transaction. A scheduled task periodically reads unpublished records, dispatches them to the integration layer, and marks them published. If the process restarts before the scheduled task runs, the record survives and is processed on the next cycle.

This gives a simple atomicity guarantee: the order and its integration task either both exist or neither does. At-least-once delivery follows naturally, which means consumers must be idempotent, duplicate messages will arrive during retries after uncertain acknowledgements.

7.2.3 Alternatives Considered

Publish after commit (dual-write). Committing the order and then publishing to the broker is the path of least resistance. The gap it creates is not theoretical: a network blip, broker restart, or process crash at that moment silently loses the integration task. Recovering requires reconciling the order table against warehouse acknowledgements by hand, which is fragile and hard to sustain operationally.

Scheduled task polling without an outbox table. A task could scan for orders in a specific status and attempt delivery without a dedicated outbox record. This sounds simpler but

effectively reconstructs outbox semantics informally. Tracking which orders have been sent and ensuring the delivery attempt and the status update are atomic requires the same care as an explicit outbox, but without the clear ownership and structure.

Relying on broker-side durability guarantees alone. Some brokers provide strong delivery guarantees once a message is accepted. The difficulty is that the producer still needs to successfully reach the broker within the same logical operation as the order commit. When the broker is temporarily unavailable at order creation time, the task is never enqueued. An outbox moves the broker dependency to the background scheduled task, which tolerates broker unavailability at commit time without affecting the customer.

7.2.4 Consequences

No paid order is silently dropped due to a transient broker or process failure. The scheduled task introduces a small propagation delay between order creation and warehouse notification, which is acceptable given the asynchronous design. The outbox table becomes integration-critical infrastructure and must be covered by database backups and operational monitoring. The publishing interval also determines the minimum propagation latency, which must remain within the bounds set by QA Scenario 1.

7.3 ADR 3 — Adapter Pattern with No Shared Database Across External Boundaries

7.3.1 Context

VerdeMart’s operational ecosystem includes at minimum a warehouse execution system and a shipping provider, with POS and ERP systems also present. As the business grows, further operational systems may be added or replaced. Each one has its own protocols, data models, and failure behaviour.

The nopCommerce database is a single schema that mixes catalog, orders, customers, inventory, shipments, configuration, and logs. Allowing external systems to query or write this schema directly would create implicit dependencies on internal table structures, making it difficult to evolve either side independently. It also creates ambiguous data ownership: if the warehouse system writes directly to the nopCommerce stock table, it is unclear which system is the authoritative source and which is a consumer.

QA Scenario 5 (Modifiability for New Operational Channels) requires that a new system can be integrated through no more than one adapter boundary, without touching checkout logic or coupling to the nopCommerce schema.

7.3.2 Decision

Each external system is connected through a dedicated adapter that translates between the internal integration contract and the system’s specific protocol, data model, and error conventions. The Warehouse, Inventory, Shipping, and Store Operations adapters are each independently replaceable. State flowing back into nopCommerce, fulfillment status, stock updates, carrier progress, arrives through events or adapter APIs and is stored as a projection with source and freshness metadata. No external system reads or writes the nopCommerce database directly.

7.3.3 Alternatives Considered

Direct database access for external systems. Giving the warehouse system or ERP read or write access to the nopCommerce database is the lowest-effort path for a small and stable set of integrations. The problem is that it couples every external system to the internal schema. Any

table rename, column change, or migration that makes sense for the commerce core becomes a cross-team coordination problem. It also makes data ownership opaque, which complicates debugging and reconciliation when systems disagree.

A single generic integration service. Routing all external events through one integration service reduces the number of deployable components. The cost is that a single service must handle the distinct failure modes, retry policies, and domain translations for every external system simultaneously. A shipping provider outage and a warehouse delay are different problems requiring different handling; merging them into one service makes both harder to reason about and monitor independently.

Plugin calls from within the nopCommerce process. nopCommerce plugins can call external systems, and this pattern is already used for payment and shipping-rate providers. For the omnichannel integration boundaries in Scenario C, however, plugins inherit the same limitations as synchronous calls: they block the request path, they have no built-in retry or dead-letter mechanism, and their availability is tied to the monolith process. They remain the right tool for internal extensibility, not for durable external coordination.

7.3.4 Consequences

Each external system has a bounded, explicit integration contract. Replacing a warehouse provider means replacing its adapter; the commerce core and the other adapters are unaffected. New channels follow the same pattern: one new adapter, one set of event subscriptions, no changes to checkout. The adapter boundary also enforces the language separation noted in Chapter 3: warehouse, POS, and carrier terminology does not leak into the core order model. The cost is that every new system needs an adapter implementation, but the scope of that work is predictable and isolated.

7.4 ADR 4 — nopCommerce Monolith Retained as the Commerce Core

7.4.1 Context

When an omnichannel scenario adds warehouse, POS, shipping, and ERP integrations, a natural design instinct is to decompose the central platform into smaller services. nopCommerce already separates concerns into layers and the plugin model gives it modularity at the code level, so the question of whether to extract services from it is worth taking seriously.

The quality attribute scenarios in Chapter 4 identify five pressures: availability, consistency, performance, recoverability, and modifiability. Looking at each one, none of them requires extracting checkout, order management, inventory, or catalog from the monolith. The existing layered architecture already provides clear seams. Breaking those capabilities into separate services would introduce distributed transaction problems and network dependencies that do not currently exist, without resolving any of the five identified pressures.

7.4.2 Decision

nopCommerce remains the commerce core and is not decomposed. Checkout, catalog, payments, customer accounts, and canonical order management stay inside the monolith. The only independently deployable component introduced by this evolution is the Integration Coordination context, which owns the outbox, retry, idempotency, and reconciliation behaviour described in ADR 2. Adapters for warehouse, shipping, inventory, and store operations sit around this layer. The monolith interacts with the Integration Coordination context through its outbox table and a publishing scheduled task, remaining decoupled from the internal design of any individual adapter.

7.4.3 Alternatives Considered

Extract order management as an independent service. Separating order creation, payment result handling, and order history into a standalone service would produce an independent deployment unit, but it removes a large body of stable, well-tested functionality from the monolith. More importantly, it would introduce a synchronous dependency between the storefront and the new order service at the point of checkout, which directly conflicts with the availability requirement in QA Scenario 1. The decomposition adds complexity without improving any of the measured quality attributes.

Extract inventory as an independent service. Given QA Scenario 2 (Consistency under Stale Inventory), a dedicated inventory service was considered. The scenario requires cross-channel stock visibility with freshness metadata and stale-state detection. The Inventory Adapter already provides this by updating projections inside nopCommerce through integration messages, extending the existing stock model rather than replacing it. Extracting inventory into a separate service would require network calls for every stock check at checkout, a new database to own the projection, and a service boundary that the adapter model renders unnecessary.

Full microservices rewrite. Decomposing nopCommerce into catalog, cart, checkout, payment, order, inventory, and notification services would require recreating a large volume of working functionality across independently operated services, coordinating distributed transactions across service boundaries, and shifting most of the project effort from architectural evolution to infrastructure. The result would be a more complex system with the same functional scope.

7.4.4 Consequences

The monolith retains its operational simplicity: one deployable unit for the full customer-facing commerce experience, with all existing capabilities, checkout, payments, catalog, admin, and plugin extensibility intact. The Integration Coordination context is the only new independently deployable component, and its scope is tightly bounded to reliability and coordination. The main limitation is that the commerce core cannot be scaled independently from the integration layer, but horizontal scaling of the combined unit is sufficient for the load levels identified in QA Scenario 3 and is not a constraint that justifies decomposition at this stage.

Chapter 8

Risk and Validation Plan

8.1 Purpose

This chapter identifies the main architectural risks in evolving the current nopCommerce implementation into the Scenario C omnichannel commerce core described in the target architecture. The current implementation is treated as the baseline: nopCommerce already supports the customer-facing commerce flow, but the planned integration layer, adapters, retry metadata, reconciliation records, and operational visibility still need to be validated before they can be claimed as implemented architecture.

The validation plan therefore focuses on the pressure required by the assignment: surrounding operational systems may be delayed, stale, unavailable, or contradictory, while the commerce core must remain useful and understandable.

8.2 Main Architectural Risks

Risk	Why it matters	Related scenarios
Surrounding systems are unavailable or delayed while check-out remains active	Warehouse or shipping failures must not cause paid orders to be lost or block later customers.	Availability under warehouse failure; recoverability after shipping outage
Inventory is stale or contradictory across online, store, and warehouse channels	The customer experience can become misleading if stock shown online no longer matches physical stock.	Consistency under stale inventory
Retry, replay, or idempotency is insufficient for asynchronous integration work	Failed integration work can be lost, repeated, or duplicated if durable retry and correlation are not designed carefully.	Availability; recoverability
Pending, degraded, failed, or recovered states are not visible to operators	A resilient system is still hard to operate if support teams cannot explain order or stock state to customers.	Availability; recoverability; observability
Integration work slows customer-facing operations	Omnichannel synchronization must not make checkout and order-status views depend on slow external systems.	Performance during omnichannel order bursts
Future adapters bypass boundaries or share the nopCommerce database	Direct coupling would make new channels harder to add and would weaken data ownership between systems.	Modifiability for new operational channels

8.3 Validation Activities

Validation activity	What it proves	Evidence to collect
Simulate a warehouse or shipping timeout during order processing	A paid order is persisted locally, marked as pending or degraded, and not lost when the external dependency fails.	Order screenshot, logs, retry record, timestamps
Simulate a delayed stock update from a store or warehouse system	The system can detect stale stock, expose the state, and create a reconciliation path instead of assuming immediate consistency.	Before and after stock views, discrepancy or reconciliation record
Verify retry, replay, and idempotency behavior for failed integration work	Repeated attempts do not create duplicate fulfillment or shipping actions, and failed work can be retried after recovery.	Retry attempts, correlation identifier, duplicate-prevention result
Verify support and operations visibility	Operators can distinguish normal, pending, degraded, failed, and recovered work without inspecting internal database tables.	Support or admin view screenshots, status transitions
Run a small order-burst check while background synchronization is active	Customer-facing checkout and order-status reads remain responsive while asynchronous work is queued or processed.	Latency measurements, queued work count, test notes
Review integration boundaries	Surrounding systems use adapters, APIs, or events and do not read or write the nopCommerce database directly.	Architecture review notes, code or configuration references

8.4 Feasibility Spike Focus

The feasibility spike should target the riskiest part of the target architecture: durable asynchronous order propagation from nopCommerce to a surrounding operational capability when that capability is slow or unavailable. The spike can use a simulator or stub for the warehouse or shipping dependency if a real system is not available.

The spike should start from a placed order or an equivalent order-created event. The external adapter should then be made unavailable or slow. The expected result is that the commerce core keeps the order, records pending integration work, exposes a degraded or awaiting-confirmation state, and later retries successfully when the dependency recovers. Chapter 10 should contain the concrete implementation details and evidence for this spike.

Chapter 9

Evolution Roadmap

The evolution roadmap describes how VerdeMart moves from the current nopCommerce baseline to the target omnichannel commerce core. The approach is incremental: each phase delivers a testable, runnable capability without taking the commerce core offline or rewriting checkout.

9.1 Implementation Order

Figure 9.1 shows the four phases in sequence and highlights what must coexist during each transition. Phase 1 is the prerequisite for everything else: no adapter can be built without a working outbox. Phases 3 and 4 both depend on Phase 2 having produced observable integration states, and Phase 4 draws on all earlier phases to build the operational visibility layer.

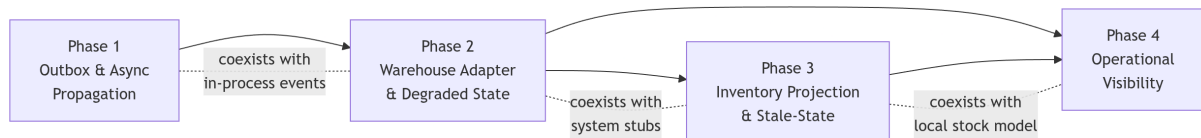


Figure 9.1: Implementation phases, ordering, and coexistence constraints.

9.1.1 Phase 1 — Outbox and Asynchronous Propagation

An outbox table is added to the nopCommerce database. Order placement writes an integration task in the same transaction as the order, and a scheduled task publishes unpublished records to the integration layer. This phase has no dependency on any external system; the publication destination can be a stub or log sink. What matters is proving the atomicity guarantee and the publishing lifecycle before any adapter is built on top.

During this phase, the outbox coexists with the existing in-process event bus. The `OrderPlacedEvent` continues to fire for internal reactions such as cache invalidation and email notifications. The outbox adds a durable external path alongside it, not in place of it.

9.1.2 Phase 2 — Warehouse Adapter and Degraded-State Handling

With a working outbox, the Warehouse Adapter is built and connected. This phase introduces the retry policy, idempotency key handling, and the integration state model: pending, sent, accepted, delayed, failed, and recovered. A warehouse simulator stands in for the real system and can be made slow or unavailable on demand, which is also how the mandatory pressure point from the scenario is demonstrated.

The shipping adapter follows the same pattern and can be introduced in parallel with this phase. System stubs coexist with the adapters throughout; real warehouse or carrier systems are not required to validate the architecture.

9.1.3 Phase 3 — Inventory Projection and Stale-State Visibility

The existing nopCommerce stock model is extended with source and freshness metadata. The Inventory Adapter receives stock updates from warehouse and store systems and updates the projection. When stock data is stale or conflicting, the model exposes a detectable state rather than silently serving an outdated number. Detected conflicts are recorded for reconciliation.

The local stock records that nopCommerce already manages are not replaced. The projection adds a visibility layer on top of them. Checkout continues to revalidate stock using the existing model; the projection is what operators and the storefront use to communicate freshness and discrepancy state.

9.1.4 Phase 4 — Operational Visibility

The operational read model aggregates the integration states produced in Phases 2 and 3 and exposes them through the nopCommerce admin area or an equivalent support-facing view. This phase introduces no new integration logic. Its purpose is to make earlier states visible and actionable: which orders are awaiting warehouse confirmation, which fulfillment requests have failed, and which inventory records are pending reconciliation.

Chapter 10

Feasibility Spike

10.1 What Was Tested

10.2 What Worked

10.3 Evidence